Lecture 6: Scheduling

CS211 - Operating systems

Spring 2026

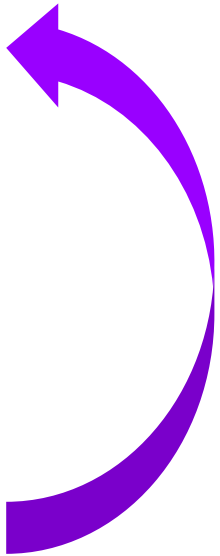
Kernel operation

Kernel operation

1. Check devices
2. Initialize “first process”
3. while device interrupts pending
    - handle device interrupts
4. while system calls pending
    - handle system calls
5. if run queue is non-empty
    - select process and switch to it
6. Otherwise
    - wait for device interrupt

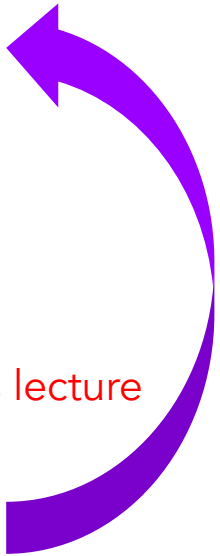
Kernel operation

1. Check devices
2. Initialize “first process”
3. while device interrupts pending
    - handle device interrupts
4. while system calls pending
    - handle system calls
5. if run queue is non-empty
    - select process and switch to it
6. Otherwise
    - wait for device interrupt



Kernel operation

1. Check devices
2. Initialize “first process”
3. while device interrupts pending
   - handle device interrupts
4. while system calls pending
   - handle system calls
5. if run queue is non-empty
   - select process and switch to it
6. Otherwise
   - wait for device interrupt



This lecture

Schedulers in an OS

- CPU scheduler
  - selects a process to run from the run queue (this lecture)
- Disk scheduler
  - selects next read/write operation
- Network scheduler
  - selects next packet to send or process
- Page replacement scheduler
  - selects page to evict

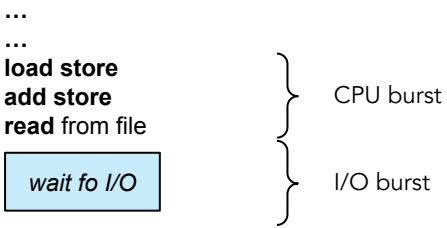
CPU bursts

CPU bursts

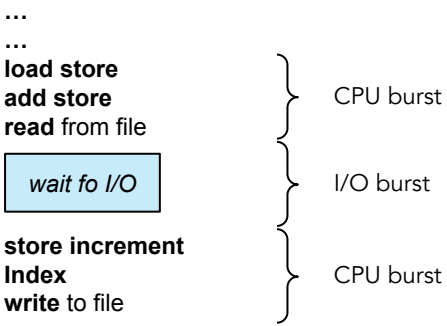
...
...
load store
add store
read from file

} CPU burst

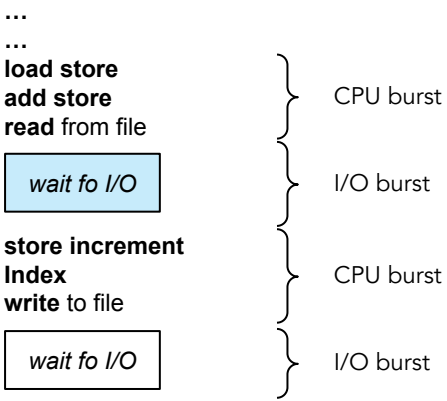
CPU bursts


...
...
load store
add store
read from file
wait fo I/O
CPU burst
I/O burst

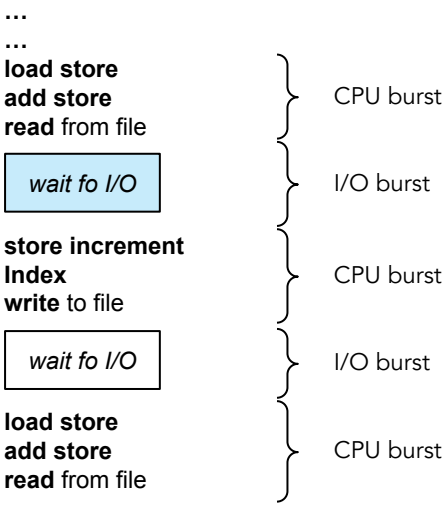
CPU bursts


...
...
load store
add store
read from file
wait fo I/O
store increment
Index
write to file
CPU burst
I/O burst
CPU burst

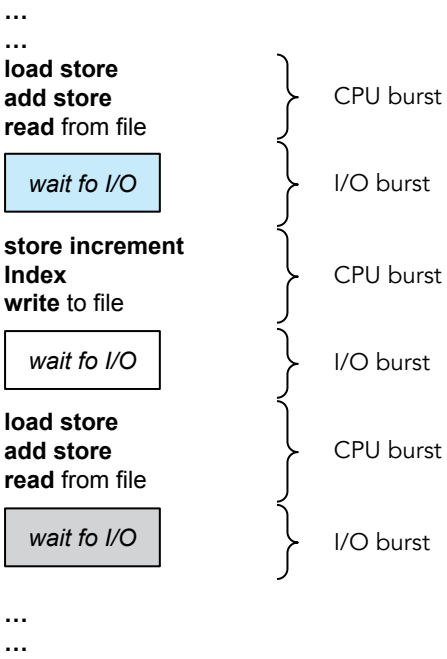
CPU bursts


...
...
load store
add store
read from file
wait fo I/O
store increment
Index
write to file
wait fo I/O
CPU burst
I/O burst
CPU burst
I/O burst

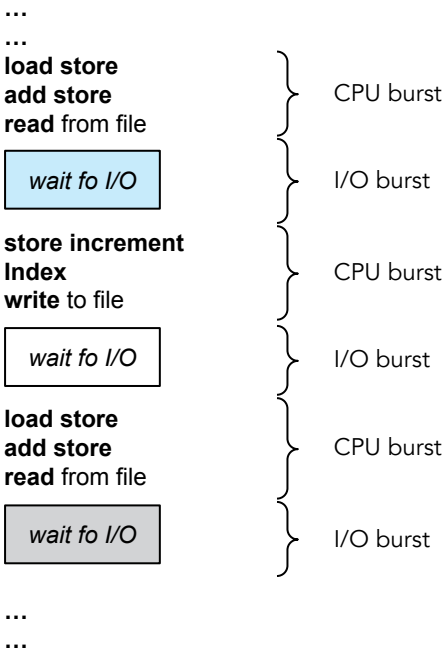
CPU bursts

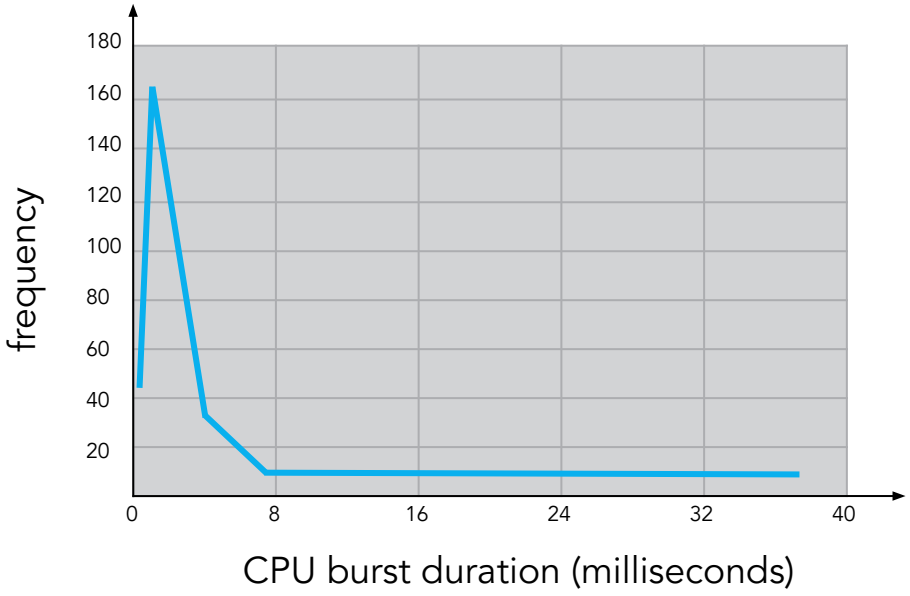

...
...
load store
add store
read from file
wait fo I/O
store increment
Index
write to file
wait fo I/O
load store
add store
read from file
CPU burst
I/O burst
CPU burst
I/O burst
CPU burst

CPU bursts


...
...
load store
add store
read from file
wait fo I/O
store increment
Index
write to file
wait fo I/O
load store
add store
read from file
wait fo I/O
...
...
CPU burst
I/O burst
CPU burst
I/O burst
CPU burst
I/O burst

CPU bursts


...
...
load store
add store
read from file
wait fo I/O
store increment
Index
write to file
wait fo I/O
load store
add store
read from file
wait fo I/O
...
...
CPU burst
I/O burst
CPU burst
I/O burst
CPU burst
I/O burst


frequency
180
160
140
120
100
80
60
40
20
0
8
16
24
32
40
CPU burst duration (milliseconds)

Job characteristics & scheduling metrics

Job characteristics & scheduling metrics

- Job: A task that needs a period of CPU time

Job characteristics & scheduling metrics

- Job: A task that needs a period of CPU time
  - Arrival time + completion deadline
  - Execution time

Job characteristics & scheduling metrics

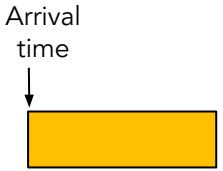
- Job: A task that needs a period of CPU time
  - Arrival time + completion deadline
    - We know
  - Execution time
    - We can only predict

Job characteristics & scheduling metrics

- Job: A task that needs a period of CPU time
  - Arrival time + completion deadline
    - We know
  - Execution time
    - We can only predict

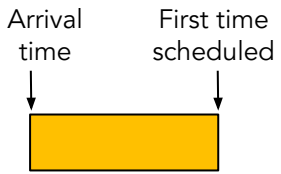
Job characteristics & scheduling metrics

- Job: A task that needs a period of CPU time
  - Arrival time + completion deadline
    - We know
  - Execution time
    - We can only predict



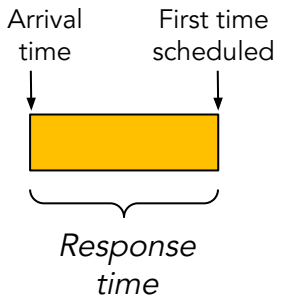
Job characteristics & scheduling metrics

- Job: A task that needs a period of CPU time
  - Arrival time + completion deadline
    - We know
  - Execution time
    - We can only predict



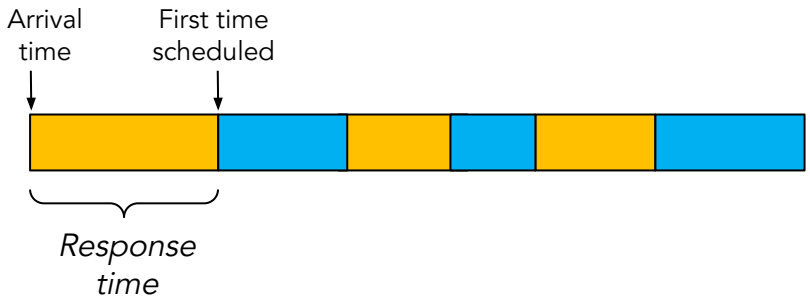
Job characteristics & scheduling metrics

- Job: A task that needs a period of CPU time
  - Arrival time + completion deadline
    - We know
  - Execution time
    - We can only predict



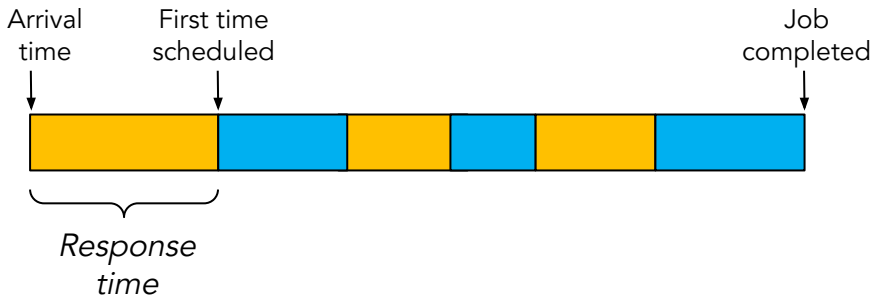
Job characteristics & scheduling metrics

- Job: A task that needs a period of CPU time
  - Arrival time + completion deadline
    - We know
  - Execution time
    - We can only predict



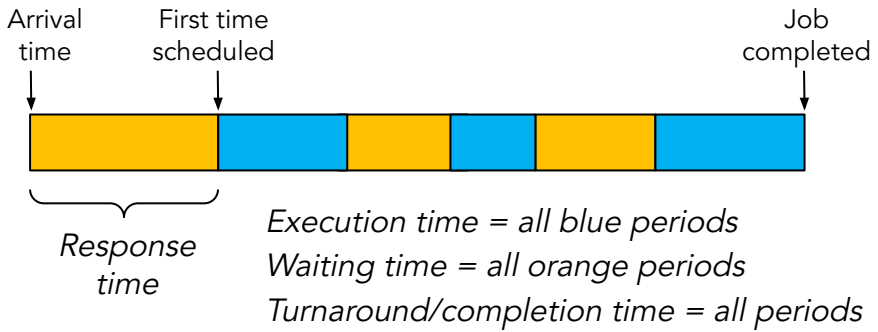
Job characteristics & scheduling metrics

- Job: A task that needs a period of CPU time
  - Arrival time + completion deadline
    - We know
  - Execution time
    - We can only predict



Job characteristics & scheduling metrics

- Job: A task that needs a period of CPU time
  - Arrival time + completion deadline
    - We know
  - Execution time
    - We can only predict



Job characteristics & scheduling metrics

- Job: A task that needs a period of CPU time
  - Arrival time + completion deadline
    - We know
  - Execution time
    - We can only predict

Execution time = all blue periods
Waiting time = all orange periods
Turnaround/completion time = all periods

- Key scheduling metrics
  - Response time
    - How quickly the system begins executing a job?
  - Throughput
    - How many jobs are completed per second?
  - Fairness
    - Ensure that each process gets a reasonable share of CPU time
  - Predictability
    - How consistent? (i.e low variance in completion time for repeated jobs)

The perfect scheduler

1. Minimizes response time for each job
2. Minimizes turnaround time for each job
3. Maximizes overall throughput
4. Maximize fairness (e.g no starvation)
    - Starvation: The lack of progress for one job due to resources given to other jobs
5. Maximizes predictable performance
6. Maximizes utilization (i.e. keeps all devices busy)
7. Meets all deadlines
8. Has zero overhead
    - Overhead: Time lost due to switching between jobs
9. …

The perfect scheduler

1. Minimizes response time for each job
2. Minimizes turnaround time for each job
3. Maximizes overall throughput
4. Maximize fairness (e.g no starvation)
    - Starvation: The lack of progress for one job due to resources given to other jobs
5. Maximizes predictable performance
6. Maximizes utilization (i.e. keeps all devices busy)
7. Meets all deadlines
8. Has zero overhead
    - Overhead: Time lost due to switching between jobs
9. …

First In First Out (FIFO)

First In First Out (FIFO)

Processes (jobs)	Execution time
P_1	12
P_2	3
P_3	3

First In First Out (FIFO)

Processes (jobs)	Execution time
P_1	12
P_2	3
P_3	3

Job arrival order?

First In First Out (FIFO)

Processes (jobs)	Execution time
P_1	12
P_2	3
P_3	3

Job arrival order?

Scenario 1: P_1, P_2, P_3

First In First Out (FIFO)

Processes (jobs)	Execution time
P_1	12
P_2	3
P_3	3

Job arrival order?

Scenario 1: P_1, P_2, P_3

First In First Out (FIFO)

Processes (jobs)	Execution time
P_1	12
P_2	3
P_3	3

Job arrival order?

Scenario 1: P_1, P_2, P_3

Average waiting time: (0 + 12 + 15)/3 = 9
Average completion time: (12 + 15 + 18)/3 = 15

First In First Out (FIFO)

Processes (jobs)	Execution time
P_1	12
P_2	3
P_3	3

Job arrival order?

Scenario 1: P_1, P_2, P_3

Average waiting time: (0 + 12 + 15)/3 = 9
Average completion time: (12 + 15 + 18)/3 = 15

Scenario 2: P_2, P_3, P_1

First In First Out (FIFO)

Processes (jobs)	Execution time
P_1	12
P_2	3
P_3	3

Job arrival order?

Scenario 1: P_1, P_2, P_3

Average waiting time: (0 + 12 + 15)/3 = 9
Average completion time: (12 + 15 + 18)/3 = 15

Scenario 2: P_2, P_3, P_1

First In First Out (FIFO)

Processes (jobs)	Execution time
P_1	12
P_2	3
P_3	3

Job arrival order?

Scenario 1: P_1, P_2, P_3

Average waiting time: (0 + 12 + 15)/3 = 9
Average completion time: (12 + 15 + 18)/3 = 15

Scenario 2: P_2, P_3, P_1

Average waiting time: (0 + 3 + 6)/3 = 3
Average completion time: (3 + 6 + 18)/3 = 9

First In First Out (FIFO)

Processes (jobs)	Execution time
P_1	12
P_2	3
P_3	3

Job arrival order?

Scenario 1: P_1, P_2, P_3

Average waiting time: (0 + 12 + 15)/3 = 9
Average completion time: (12 + 15 + 18)/3 = 15

Scenario 2: P_2, P_3, P_1

Average waiting time: (0 + 3 + 6)/3 = 3
Average completion time: (3 + 6 + 18)/3 = 9

Pros/Cons?

Round Robin (RR)

Round Robin (RR)

- Each job allowed to run for a period of time (aka “quantum”)

Round Robin (RR)

- Each job allowed to run for a period of time (aka “quantum”)
  - Example:

Processes (jobs)	Execution time
P_1	12
P_2	3
P_3	3

… and job arrival order is P_1, P_2, P_3

Round Robin (RR)

- Each job allowed to run for a period of time (aka "quantum")
  - Example:

Processes (jobs)	Execution time
P_1	12
P_2	3
P_3	3

… and job arrival order is P_1, P_2, P_3

Quantum = 3:

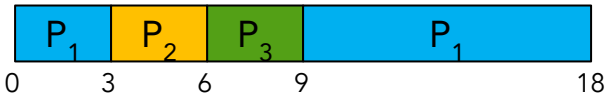
Round Robin (RR)

- Each job allowed to run for a period of time (aka "quantum")
  - Example:

Processes (jobs)	Execution time
P_1	12
P_2	3
P_3	3

… and job arrival order is P_1, P_2, P_3

Quantum = 3:



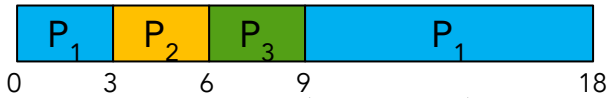
Round Robin (RR)

- Each job allowed to run for a period of time (aka "quantum")
  - Example:

Processes (jobs)	Execution time
P_1	12
P_2	3
P_3	3

… and job arrival order is P_1, P_2, P_3

Quantum = 3:



Average waiting time: (3 + 6 + 6)/3 = 5
Average completion time: (18 + 6 + 9)/3 = 11

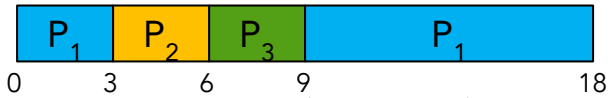
Round Robin (RR)

- Each job allowed to run for a period of time (aka "quantum")
  - Example:

Processes (jobs)	Execution time
P_1	12
P_2	3
P_3	3

… and job arrival order is P_1, P_2, P_3

Quantum = 3:



Average waiting time: (3 + 6 + 6)/3 = 5
Average completion time: (18 + 6 + 9)/3 = 11

Quantum = 6

Round Robin (RR)

- Each job allowed to run for a period of time (aka "quantum")
  - Example:

Processes (jobs)	Execution time
P_1	12
P_2	3
P_3	3

… and job arrival order is P_1, P_2, P_3

Quantum = 3:

Average waiting time: (3 + 6 + 6)/3 = 5
Average completion time: (18 + 6 + 9)/3 = 11

Quantum = 6

Round Robin (RR)

- Each job allowed to run for a period of time (aka "quantum")
  - Example:

Processes (jobs)	Execution time
P_1	12
P_2	3
P_3	3

… and job arrival order is P_1, P_2, P_3

Quantum = 3:

Average waiting time: (3 + 6 + 6)/3 = 5
Average completion time: (18 + 6 + 9)/3 = 11

Quantum = 6

Average waiting time: (6 + 6 + 9)/3 = 7
Average completion time: (18 + 9 + 12)/3 = 13

Round Robin (RR)

- Each job allowed to run for a period of time (aka "quantum")
  - Example:

Processes (jobs)	Execution time
P_1	12
P_2	3
P_3	3

… and job arrival order is P_1, P_2, P_3

Quantum = 3:

Average waiting time: (3 + 6 + 6)/3 = 5
Average completion time: (18 + 6 + 9)/3 = 11

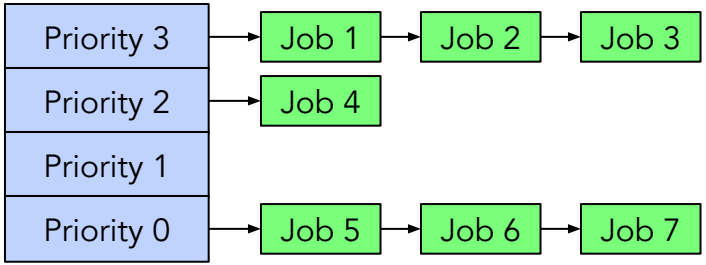
Quantum = 6

Average waiting time: (6 + 6 + 9)/3 = 7
Average completion time: (18 + 9 + 12)/3 = 13

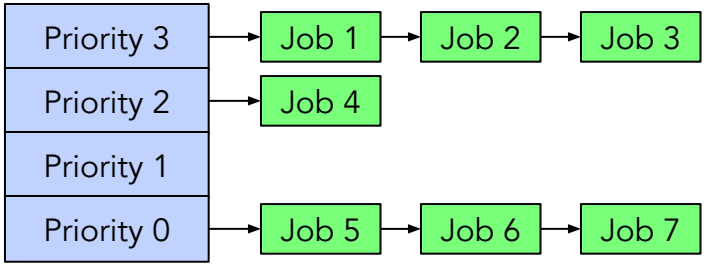
Pros/Cons?

Priority scheduling

Priority scheduling

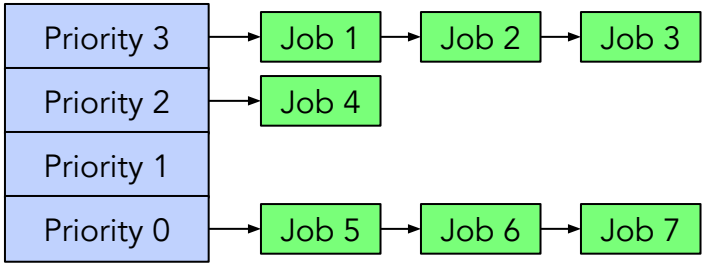


Priority scheduling



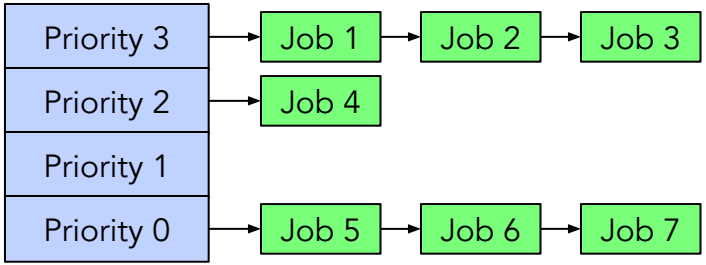
- Always execute highest-priority runable jobs to completion
  - Jobs with the same priority level can be scheduled by RR

Priority scheduling



- Always execute highest-priority runable jobs to completion
  - Jobs with the same priority level can be scheduled by RR
- Problems:
  - Starvation
    - Lower priority jobs don't get to run because higher priority jobs
  - Deadlock
    - Low priority task has lock needed by high-priority task! (aka "Priority Inversion")

Priority scheduling



- Always execute highest-priority runable jobs to completion
  - Jobs with the same priority level can be scheduled by RR
- Problems:
  - Starvation
    - Lower priority jobs don't get to run because higher priority jobs
  - Deadlock
    - Low priority task has lock needed by high-priority task! (aka "Priority Inversion")
- How to fix problems?
  - Give each queue (of each priority level) some fraction of the CPU (aka "Fair sharing")
  - Adjust/update the priority based on heuristics about interactivity, locking, burst behavior, etc

If we know the future …

Shortest job first (SJF)

Shortest job first (SJF)

- Run jobs has least amount of computation to do

Shortest job first (SJF)

- Run jobs has least amount of computation to do
- Example
  - FIFO

Shortest job first (SJF)

- Run jobs has least amount of computation to do
- Example
  - FIFO

Average waiting time: (0 + 12 + 15)/3 = 9
Average completion time: (12 + 15 + 18)/3 = 15

Shortest job first (SJF)

- Run jobs has least amount of computation to do
- Example
  - FIFO

Average waiting time: (0 + 12 + 15)/3 = 9
Average completion time: (12 + 15 + 18)/3 = 15

  - SJF

Shortest job first (SJF)

- Run jobs has least amount of computation to do
- Example
  - FIFO

Average waiting time: (0 + 12 + 15)/3 = 9
Average completion time: (12 + 15 + 18)/3 = 15

  - SJF

Shortest job first (SJF)

- Run jobs has least amount of computation to do
- Example
  - FIFO

Average waiting time: (0 + 12 + 15)/3 = 9
Average completion time: (12 + 15 + 18)/3 = 15

  - SJF

Average waiting time: (0 + 3 + 6)/3 = 3
Average completion time: (3 + 6 + 18)/3 = 9

Shortest remaining time first (SRTF)

Shortest remaining time first (SRTF)

- Preemptive version of SJF

Shortest remaining time first (SRTF)

- Preemptive version of SJF
  - If job arrives and has a shorter time to completion than the remaining time on the current job, immediately preempt CPU

Earliest deadline first (EDF)

Earliest deadline first (EDF)

- Each job is assigned a priority based on how close the deadline is

Choosing the right scheduler

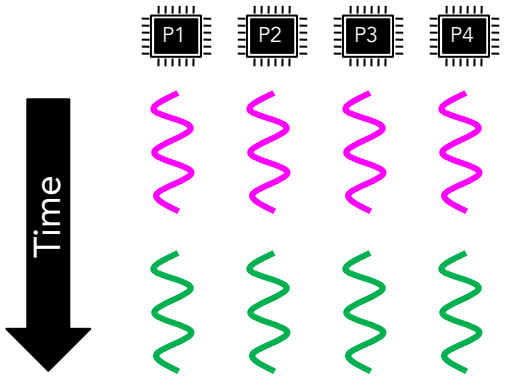
I care about	... then I choose
CPU Throughput	???
Avg. Response Time	???
I/O Throughput	???
Fairness (CPU Time)	???
Meeting Deadlines	???
Favoring Important Tasks	???

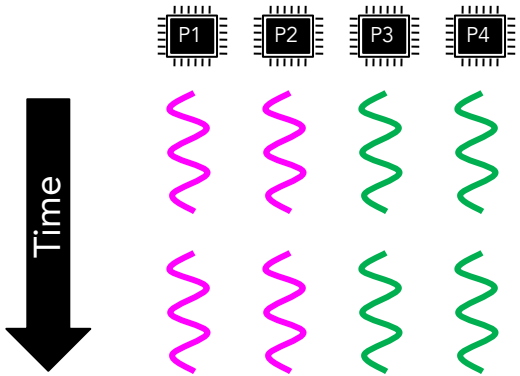
Choosing the right scheduler

I care about	... then I choose
CPU Throughput	FCFS
Avg. Response Time	SRTF
I/O Throughput	SRTF
Fairness (CPU Time)	Round Robin, Fair Sharing
Meeting Deadlines	EDF
Favoring Important Tasks	Priority scheduling

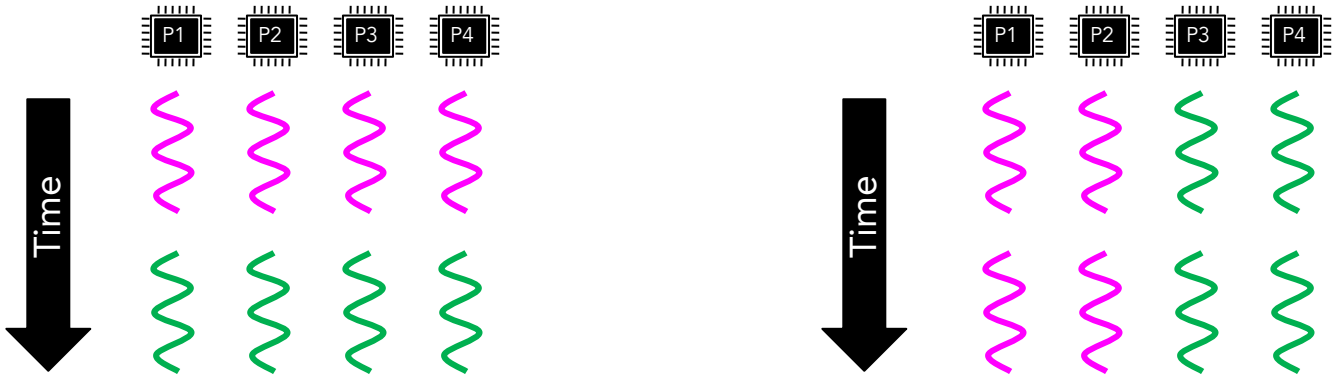
Multi-core scheduling

Multi-core scheduling





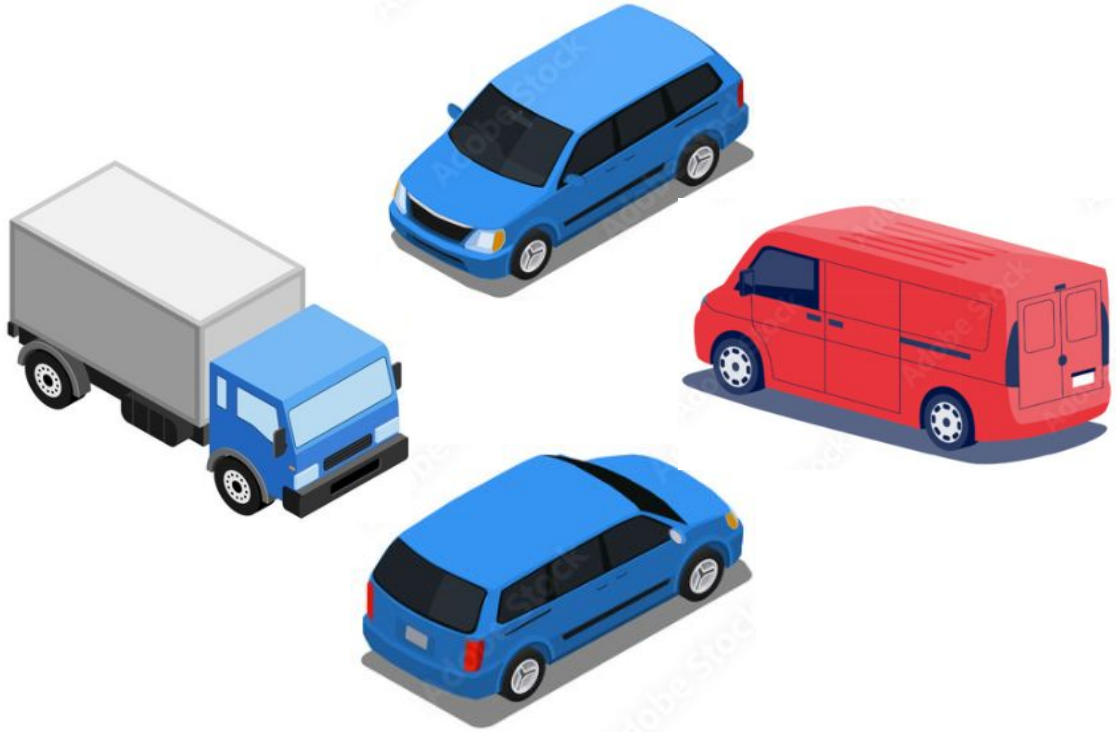
Multi-core scheduling



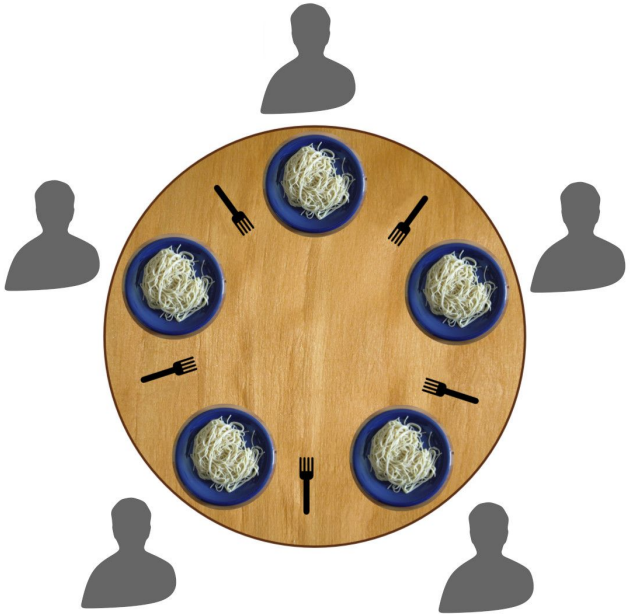
- Desirables
  - Balance load
    - Each job should get approximately the same amount of CPU, no matter what core it runs on
  - Scheduling affinity
    - Avoid moving processes between cores to avoid wasting cache content (L1, TLB, etc.)

Deadlock

Deadlock



Dining philosophers problem



Spaghetti must be eaten with two forks, but each philosopher may hold only one fork while waiting for the other

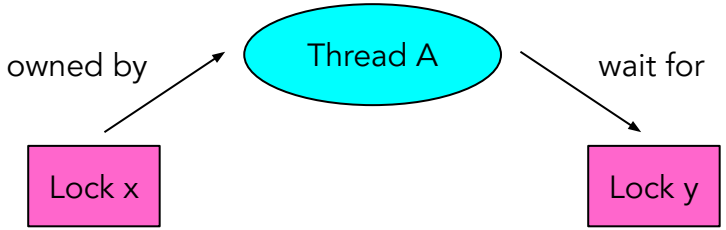
Deadlock because of locks

Deadlock because of locks

Thread A:	Thread B:
`x.acquire()`	`y.acquire()`
`y.acquire()`	`x.acquire()`
…	…
`y.release()`	`x.release()`
`x.release()`	`y.release()`

Deadlock because of locks

Thread A:	Thread B:
`x.acquire()`	`y.acquire()`
`y.acquire()`	`x.acquire()`
…	…
`y.release()`	`x.release()`
`x.release()`	`y.release()`



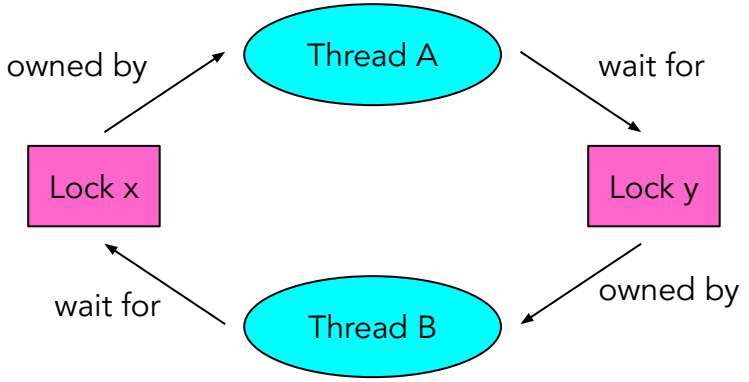
Deadlock because of locks

Thread A:

```
x.acquire()
y.acquire()
…
y.release()
x.release()
```

Thread B:

```
y.acquire()
x.acquire()
…
x.release()
y.release()
```



Other types of deadlock

- Threads often block waiting for resources
  - Locks
  - Terminals
  - Printers
  - CD drives
  - Memory
- Threads often block waiting for other threads
  - Pipes
  - Sockets

Four requirements for deadlock occurrence

Four requirements for deadlock occurrence

1. Mutual exclusion
    - Only one thread at a time can use a resource

Four requirements for deadlock occurrence

1. Mutual exclusion
    - Only one thread at a time can use a resource
2. Hold and wait
    - Thread holding at least one resource is waiting for additional resources held by other threads

Four requirements for deadlock occurrence

1. Mutual exclusion
    - Only one thread at a time can use a resource
2. Hold and wait
    - Thread holding at least one resource is waiting for additional resources held by other threads
3. No preemption
    - Resources are released only voluntarily by the thread holding the resource, after thread is finished with it

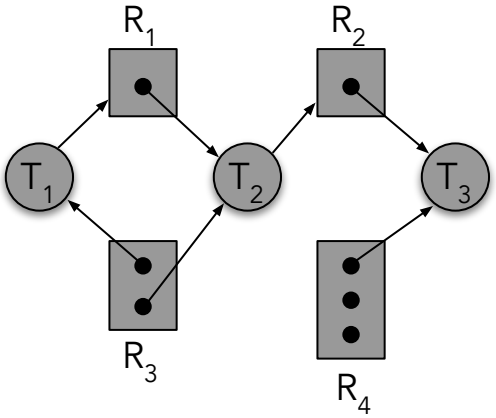
Four requirements for deadlock occurrence

1. Mutual exclusion
   - Only one thread at a time can use a resource
2. Hold and wait
   - Thread holding at least one resource is waiting for additional resources held by other threads
3. No preemption
   - Resources are released only voluntarily by the thread holding the resource, after thread is finished with it
4. Circular wait
   - There exists a set {T1, …, Tn} of waiting threads
     - T1 is waiting for a resource that is held by T2
     - T2 is waiting for a resource that is held by T3
     - …
     - Tn is waiting for a resource that is held by T1

Resource allocation graph

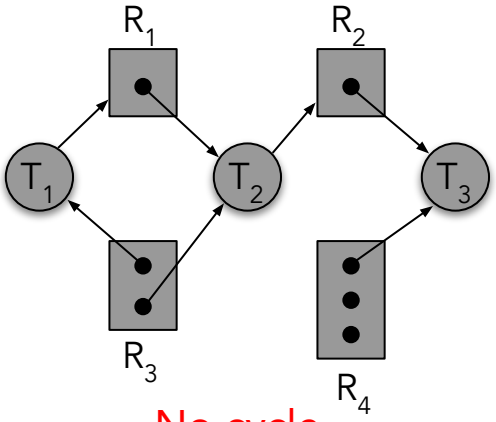
Resource allocation graph

- T_i is the i^{th} thread
- R_j is the j^{th} resource
- $T_i \rightarrow R_j$
  - The i^{th} thread requests a copy of the j^{th} resource
- $R_j \rightarrow T_i$
  - A copy of the j^{th} resource is allocated to the i^{th} thread



Resource allocation graph

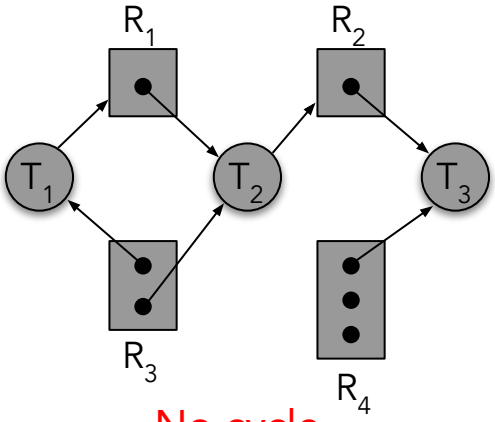
- T_i is the i^{th} thread
- R_j is the j^{th} resource
- $T_i \rightarrow R_j$
  - The i^{th} thread requests a copy of the j^{th} resource
- $R_j \rightarrow T_i$
  - A copy of the j^{th} resource is allocated to the i^{th} thread



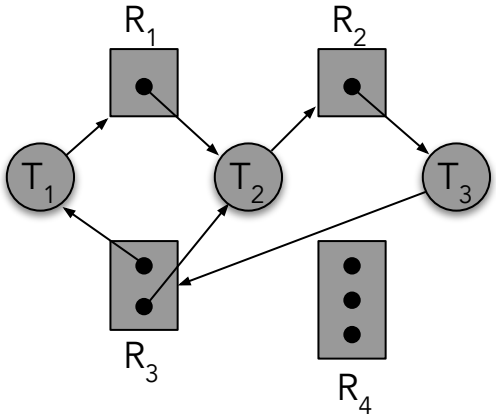
No cycle

Resource allocation graph

- T_i is the i^{th} thread
- R_j is the j^{th} resource
- $T_i \rightarrow R_j$
  - The i^{th} thread requests a copy of the j^{th} resource
- $R_j \rightarrow T_i$
  - A copy of the j^{th} resource is allocated to the i^{th} thread

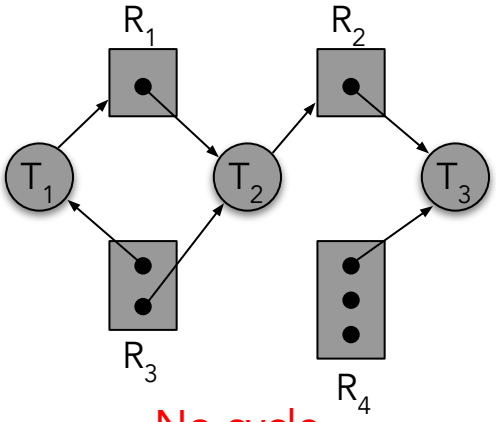


No cycle

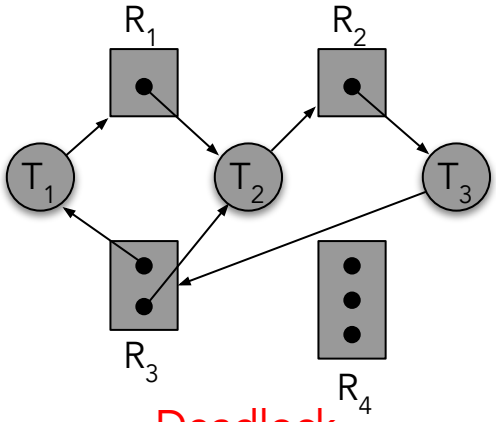


Resource allocation graph

- T_i is the i^{th} thread
- R_j is the j^{th} resource
- $T_i \rightarrow R_j$
  - The i^{th} thread requests a copy of the j^{th} resource
- $R_j \rightarrow T_i$
  - A copy of the j^{th} resource is allocated to the i^{th} thread



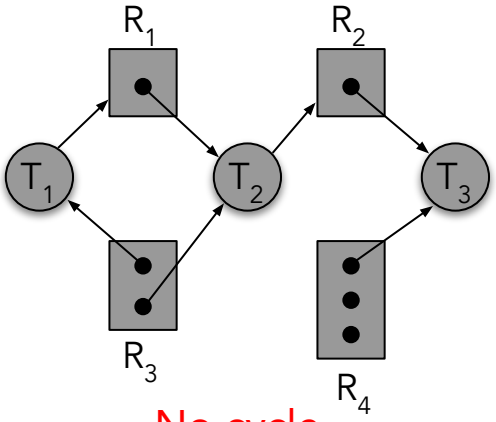
No cycle



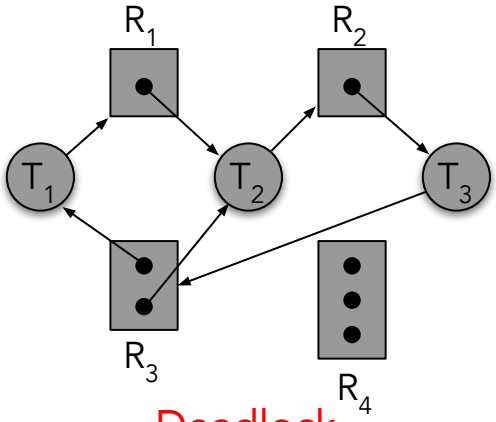
Deadlock

Resource allocation graph

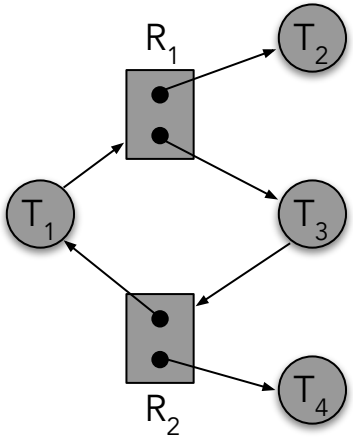
- T_i is the i^{th} thread
- R_j is the j^{th} resource
- $T_i \rightarrow R_j$
  - The i^{th} thread requests a copy of the j^{th} resource
- $R_j \rightarrow T_i$
  - A copy of the j^{th} resource is allocated to the i^{th} thread



No cycle

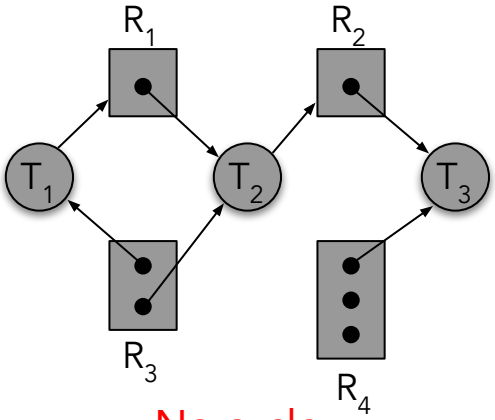


Deadlock

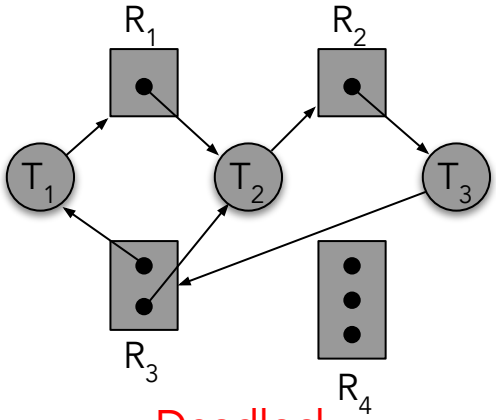


Resource allocation graph

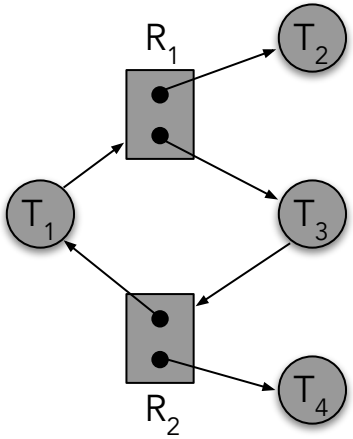
- T_i is the i^{th} thread
- R_j is the j^{th} resource
- $T_i \rightarrow R_j$
  - The i^{th} thread requests a copy of the j^{th} resource
- $R_j \rightarrow T_i$
  - A copy of the j^{th} resource is allocated to the i^{th} thread



No cycle



Deadlock



Cycle but no deadlock

Deadlock detection algorithm

Deadlock detection algorithm

Inputs:

FreeResources: a vector of non-negative integers representing the quantities of each resource type that are not being used

Request[t]: a vector of non-negative integers representing the quantities of each resource type requested by thread t

Alloc[t]: a vector of non-negative integers representing the quantities of each resource type held by thread t

Output: The deadlock occurs or not

```
Avail = FreeResources
add all threads to UNFINISHED
do {
    done = true
    for each thread t in UNFINISHED {
        if (Request[t] <= Avail) {
            remove t from UNFINISHED
            Avail += Alloc[t]
            done = false
    }
}
if UNFINISHED is not empty
    return deadlocked
```

Techniques for addressing deadlock

Techniques for addressing deadlock

- Deadlock prevention
  - write your code in a way that it isn't prone to deadlock
- Deadlock recovery
  - let deadlock happen, and then figure out how to recover from it
- Deadlock avoidance
  - Dynamically delay resource requests so deadlock doesn't happen
- Deadlock denial
  - ignore the possibility of deadlock

Summary

- Scheduling policies
  - FIFO
  - Round robin
  - Priority scheduling
  - Fair Sharing
  - SJF
  - SRTF
  - EDF
- Deadlock
  - What causes that deadlock?
  - How to detect the deadlock?
  - How to deal with deadlocks?